

Futures Stop-Loss Engine — Unified Specification

Version: 2.0

Type: Conceptual and Deterministic Hybrid

Status: Production-Ready and Review-Ready

PART A — CONCEPTUAL DESIGN

1. Objective

The objective of this system is to design and implement a deterministic, automated stop-loss engine for futures positions that ensures consistent risk control, execution reliability, and adaptability to market conditions.

The system must satisfy the following core goals:

1.1 Minimize Downside Risk

The system shall define and enforce a maximum permissible loss per position at the time of trade entry and ensure that:

- The realized loss does not materially exceed the predefined risk, except in extreme conditions such as gap moves or illiquidity
- Risk exposure is quantified, bounded, and continuously monitored
- The stop-loss acts as a hard risk boundary and not a soft signal

The stop-loss must function as a strict risk containment mechanism. Every position must always have a clearly defined exit level.

1.2 Avoid Premature Exits (Market Noise Resistance)

The system shall prevent exits caused by short-term price fluctuations that do not represent a structural change in the trade direction by:

- Using volatility-adjusted thresholds based on ATR
- Anchoring stop levels to market structure such as swing highs and lows
- Ignoring incomplete or unstable price data such as live or forming candles

The intent is to exit only when the trade thesis is invalidated, not when the market exhibits normal noise.

1.3 Adapt to Volatility and Market Structure

The system shall dynamically adjust stop placement based on:

- Volatility conditions, including expansion and contraction of ATR
- Market structure derived from recent price action
- Real-time market data such as last traded price and spread

This ensures that:

- Stops widen during high volatility to avoid premature exits
- Stops tighten during low volatility to improve capital efficiency
- The system remains robust across different instruments and market regimes

A static stop is insufficient. The system must scale its behavior with changing market conditions.

1.4 Guarantee Execution Safety

The system shall ensure that once a stop condition is reached, the position is reliably exited by:

- Using execution-aware mechanisms such as GTT orders with trigger and limit buffers
- Implementing fallback mechanisms such as market orders in case of failure or partial execution
- Continuously verifying that every open position is protected by an active stop
- The system shall operate on an end-of-day risk management cycle, where stop-loss levels are computed after market close using completed market data and are placed for protection in the next trading session.

The system must also:

- Detect and recover from missing or failed stop orders

- Handle gap scenarios and fast-moving markets without leaving positions unprotected

A stop that fails to execute is equivalent to having no stop. Execution reliability is as important as stop accuracy.

1.5 Ensure Consistency Between Logic and Execution

The system shall maintain strict alignment between computed stop levels and actual placed orders such that:

- No discrepancy exists between intended risk and actual protection
 - All updates are traceable, verifiable, and idempotent
 - The system behaves deterministically under repeated execution
 - The system shall persist the stop lifecycle for each position from trade origination to closure, including initial stop, every accepted stop revision, associated trigger and limit prices, execution identifiers, and timestamps
 - The system shall persist trade origination timestamp and stop-state history for each position from origination to closure
 - The system shall maintain an internally persisted and continuously updated trade history to support deterministic reconstruction of position lifecycle and stop evolution
-

1.6 Provide Continuous Risk Visibility

The system shall provide real-time visibility into:

- Current risk per position
- Distance between current price and stop
- Unrealized profit or loss relative to stop level
- Historical stop progression for each open and closed position
- Initial risk, current risk, and locked-in risk over the life of the trade

Risk must not only be controlled but also continuously observable.

Summary

The system is designed to function as a deterministic risk containment and execution mechanism, ensuring that every position has a reliable, adaptive, and enforceable exit strategy under all market conditions.

2. Core Philosophy

2.1 Risk Must Only Reduce

Once a position is entered, the system shall ensure that risk does not increase at any point during the lifecycle of the trade.

This requirement prevents uncontrolled loss expansion and aligns with disciplined risk management practices.

2.2 Market Structure and Volatility

The stop-loss shall be derived using a combination of market structure and volatility.

- Market structure is represented by swing highs and swing lows and reflects the directional intent of price movement.
- Volatility is represented by the Average True Range and reflects the natural range of price fluctuations.

This combination ensures that the stop is neither excessively tight nor excessively loose.

2.3 Separation of Stop Types

The system shall maintain two distinct stop types:

- Initial Stop, which defines the maximum acceptable loss at the time of entry
- Trailing Stop, which adjusts over time to protect unrealized gains

These two components must remain logically independent to ensure predictable behavior.

2.4 Execution Reliability Over Price Precision

The system shall prioritize reliable execution of exits over achieving optimal exit prices.

In practical terms:

- A slightly suboptimal exit price is acceptable
 - A failed or missed exit is not acceptable
-

2.5 Stability Over Reactivity

The system shall avoid reacting to unstable or incomplete market data.

- Only completed candles shall be used for structural calculations
- Short-term fluctuations that do not invalidate the trade premise shall be ignored
- The engine shall not react to intraday price fluctuations during market hours. Stop computation shall be based on completed post-market data only.

This ensures that the stop-loss remains stable in choppy market conditions.

3. Key Risks Addressed

The system is designed to address the following previously identified risks:

- Misalignment between computation timeframe and execution timeframe
- Use of stale price data
- Stop values that increase risk over time
- Lack of validation before order placement
- Unreliable execution during fast market conditions

These risks are resolved through deterministic rules defined in Part B.

PART B — DETERMINISTIC SPECIFICATION

4. Core Invariants

The following rules shall always hold true:

4.1 Monotonicity

- For long positions, the stop price shall not decrease over time
 - For short positions, the stop price shall not increase over time
-

4.2 Logical Validity

- For long positions, the stop price shall always be below the current market price
 - For short positions, the stop price shall always be above the current market price
-

4.3 Minimum Risk Distance

The distance between the current price and the stop price shall always be greater than or equal to the minimum defined threshold.

5. Parameter System

5.1 ATR Configuration

- ATR Period: 14
- ATR Average Window: 20
- Base Multiplier: 2.5

Volatility Adjustment

If the current ATR is greater than or equal to twice the average ATR:

- Multiplier shall be set to 3.25

Otherwise:

- Multiplier shall remain 2.5
-

5.2 Swing Calculation

- Lookback window: 5 completed candles
 - Swing low is defined as the minimum low over the lookback window
 - Swing high is defined as the maximum high over the lookback window
-

5.3 Minimum Distance

Minimum distance shall be defined as:

- The maximum of 0.5 percent of the current price
 - And 0.5 times the ATR
-

5.4 Spread Handling

- Spread is defined as ask price minus bid price

Spread threshold shall be defined as:

- The maximum of 0.15 percent of the current price
 - And 1.5 times the tick size
-

5.5 Execution Buffer

Base buffer shall be defined as:

- The maximum of 0.1 percent of the current price
- And one tick size

If spread exceeds the threshold:

- Buffer shall be doubled

Otherwise:

- Base buffer shall be used
-

5.6 Update Threshold

A stop update shall only be applied if:

- The absolute difference between the new stop and existing stop is greater than or equal to
 - Two times the tick size
 - Or 0.05 percent of the current price
-

6. Stop Computation

6.1 Initial Stop

- For long positions, stop equals entry price minus ATR multiplied by multiplier
 - For short positions, stop equals entry price plus ATR multiplied by multiplier
-

6.2 Trailing Stop

For long positions:

- Candidate stop is the maximum of swing low and close price minus ATR multiplied by multiplier.
- New stop is the maximum of previous stop and candidate stop

For short positions:

- Candidate stop is the minimum of swing high and close price plus ATR multiplied by multiplier
 - New stop is the minimum of previous stop and candidate stop
-

7. Validation Layer

If a computed stop is invalid:

- The stop shall be adjusted to maintain minimum distance from current price
-

8. Execution Engine

8.1 Order Construction

- For long positions:
 - Trigger price equals stop
 - Limit price equals stop minus buffer
 - For short positions:
 - Trigger price equals stop
 - Limit price equals stop plus buffer
-

8.2 Idempotency

- A hash shall be generated using trigger price, limit price, and quantity
 - Orders shall only be updated if the hash differs from the last applied state
-

8.3 Retry Logic

- Up to three retries shall be attempted in case of failure
 - Retry delay shall increase progressively
-

8.4 Recovery Rules

- If no stop order exists, a new stop shall be placed immediately
- If a partial exit occurs, the remaining position shall be closed using a market order
- If price moves beyond stop without execution, an immediate market exit shall be triggered
- If broker stop state is missing, inconsistent, or unavailable, the system shall reconstruct the latest valid stop from persisted internal stop history and re-establish protection
- If trade origination or lifecycle state cannot be determined from current broker state, the system shall reconstruct it using persisted internal trade history
- If price moves beyond the stop level during the next trading session without successful stop execution, the system shall detect the failure in the next scheduled recovery cycle and initiate the configured recovery action.

8.5 Stop Execution Timing Model

The system shall compute stop-loss levels after market close and before the next trading session.

The stop engine job shall run once per trading day after market close, using only completed candle data and end-of-day market information available at the time of execution.

For end-of-day stop computation, the current price reference shall be the close price of the most recently completed end-of-day candle for the instrument.

The stop-loss orders generated by the engine shall be intended to protect the position during the next trading session.

The system shall not continuously recompute or modify stop-loss levels during market hours unless explicitly configured to operate in intraday mode.

If the stop engine is executed during market hours while operating in end-of-day mode, it shall not use live prices, intraday last-traded prices, streaming data, or partial candle data for stop computation.

In such cases, the current price reference shall remain the close price of the most recently completed end-of-day candle, which shall be the previous trading day's official close for the instrument.

Any stop-loss levels computed during market hours in end-of-day mode shall be based only on completed end-of-day data and other market information available from the prior completed trading session.

If stop-loss levels for the current trading day have already been computed and applied, any additional execution of the stop engine during market hours in end-of-day mode shall not recompute or modify those levels and shall either return the previously accepted stop state or take no action, according to system configuration.

The operational sequence shall be:

- Market closes
- End-of-day data becomes available
- Stop engine computes validated stop-loss levels
- Stop orders are placed or modified after market hours
- Broker-side stop protection remains active during the next trading session

9. Concurrency Control

- All operations shall be executed under an instrument-level lock
- Stop computation, validation, and execution shall be atomic

10. State Management

The system shall follow the following position lifecycle:

- New
- Protected
- Trailing
- Exiting
- Closed

Each state shall transition deterministically based on execution events.

The system shall maintain persistent stop-state history from the date of trade origination until trade closure.

For each accepted stop event, the system shall store:

- Position identifier
- Trade origination timestamp
- Stop calculation timestamp
- Stop type (initial or trailing)
- Entry price
- Current market price at calculation time
- ATR
- ATR average
- Applied multiplier
- Swing reference values
- Raw stop
- Validated stop
- Trigger price
- Limit price
- Quantity
- Execution action taken
- Broker trigger identifier, if applicable

The latest accepted stop shall be treated as the operational stop state, and the full sequence of prior accepted stops shall be retained for audit, recovery, and analysis.

10.1 Trade Origination and Stop Persistence

The system shall identify and persist the trade origination timestamp for each position.

Trade origination timestamp shall be defined as the timestamp at which the net position for a given instrument and side first changes from zero to non-zero for the currently open trade lifecycle.

The system shall not rely solely on the current broker positions snapshot to determine trade origination.

Where available, trade origination shall be derived from persisted internal trade records. If persisted internal trade records are not available, trade origination shall be reconstructed from broker-executed trade history and order history.

Once determined, the trade origination timestamp shall be stored as part of the internal position state and retained until the position is fully closed.

The system shall maintain stop history from trade origination through position closure.

Trade History Ingestion and Maintenance

The system shall maintain an internal trade history store for each instrument and position lifecycle.

At system initialization, historical trade data shall be ingested from broker-provided tradebook or order history and used to reconstruct position lifecycles and trade origination timestamps.

The system shall thereafter continuously update this internal trade history by ingesting newly executed trades from the broker at a defined interval.

Trade ingestion shall be incremental and idempotent, ensuring that:

- Duplicate trades are not reprocessed
- Trade ordering is preserved based on execution timestamp
- Net position is accurately reconstructed over time

The system shall derive position lifecycle boundaries by computing cumulative net quantity per instrument such that:

- A lifecycle begins when net quantity transitions from zero to non-zero
- A lifecycle ends when net quantity returns to zero

The trade origination timestamp for an open position shall be defined as the timestamp of the first trade in the current lifecycle.

The internal trade history shall be treated as the source of truth for trade origination and lifecycle reconstruction, and broker position snapshots shall be treated as execution state only.

11. Monitoring and Alerts

Alert Conditions

- Warning for invalid stop conditions
- Error for retry attempts
- Critical alert if a position is not protected

Kill Switch

The system shall halt execution if:

- More than 5 percent of positions are unprotected
- Or three consecutive execution failures occur

12. Backtesting Requirements

Method

- Walk-forward validation shall be used

Acceptance Criteria

- Maximum drawdown shall not exceed baseline by more than 5 percent
- Profit retention shall be at least 70 percent
- Stop efficiency shall be at least 60 percent
- Backtesting shall simulate end-of-day stop calculation with next-session stop effectiveness, rather than intraday dynamic stop recomputation.

13. System Integration & Safety Validation

This section defines the mandatory test scenarios required for system certification. Every implementation must pass these cases in a simulated environment before production deployment to ensure compliance with the Risk Management Specification.

13.1 Core Invariant Validation

- **VAL-01 | Long Monotonicity:** Attempt to lower an existing Stop-Loss price for a Long position.
 - *Expected Outcome:* **Reject.** Engine must block the update and log a "Monotonicity Violation."
- **VAL-02 | Short Monotonicity:** Attempt to raise an existing Stop-Loss price for a Short position.
 - *Expected Outcome:* **Reject.** Engine must block the update and log a "Monotonicity Violation."
- **VAL-03 | Hash Suppression:** Send an update request where price, quantity, and side are identical to the current active state.
 - *Expected Outcome:* **Ignore.** No API call to the broker shall be generated; internal state remains unchanged.

13.2 Volatility Logic (ATR Scaling)

- **VAL-04 | Regime Shift (High):** Simulate $ATR_{\text{current}} \geq 2.0 \times ATR_{\text{avg}}$.
 - *Expected Outcome:* Stop distance must scale to the **3.25x ATR** multiplier automatically.
- **VAL-05 | Lookback Integrity:** Provide a data stream with a price spike in the current candle.
 - *Expected Outcome:* Engine must ignore the current candle and calculate swing based on the **5 previously completed** candles.

13.3 Critical Recovery (Failure Modes)

- **VAL-06 | Orphaned State:** Manually delete the Stop order via the broker UI while the position is open.
 - *Expected Outcome:* **Auto-Reconstruction.** Engine must detect the missing order and re-place it using persisted state.
- **VAL-07 | Slippage/Gap-Over:** Simulate a price gap that exceeds the Stop Trigger Price instantly.
 - *Expected Outcome:* **Immediate Market Exit.** Engine must bypass the Limit Stop and issue a Market Order to close the position.
- **VAL-08 | Partial Fill:** Simulate a partial exit (e.g., scale-out) that reduces the position size.
 - *Expected Outcome:* **Qty Sync.** Engine must immediately revise the Stop-Loss quantity to match the remaining exposure.

13.4 Safety & Concurrency

- **VAL-09 | Race Condition:** Issue two rapid-fire price updates for the same instrument within <10ms.
 - *Expected Outcome:* **Sequential Execution.** Instrument lock must ensure the first update completes before the second is evaluated.

- **VAL-10 | Global Kill Switch:** Activate the System-Wide Kill Switch.
 - *Expected Outcome: Purge.* All active stop orders must be cancelled and the engine must enter a "Read-Only" state.

PART C — LINK BETWEEN CONCEPT AND IMPLEMENTATION

The system ensures alignment between conceptual intent and implementation as follows:

- Risk reduction is enforced through monotonic stop rules
- Noise resistance is achieved through ATR and completed candle usage
- Execution reliability is ensured through buffers and fallback market orders
- Volatility adaptation is implemented through ATR-based multiplier adjustments
- Update frequency is controlled through defined thresholds